Summary: Using SymPy for a Laplace Equation Project

1 Project goal

The goal is to let the user provide a complex function $f(z)$, then automatically:

1. parse it as a SymPy expression,
2. substitute $z = x + iy$,
3. extract the real part
$$u(x, y) = \Re(f(x + iy)),$$
4. compute boundary conditions on the square $[0, 1] \times [0, 1]$,
5. build a callable numeric function $u(x, y)$,
6. optionally verify that u is harmonic, so it can be used as an exact solution of Laplace's equation.

2 Core mathematical idea

If $f(z)$ is holomorphic, then its real part $u(x, y) = \Re(f(z))$ is harmonic, which means:

$$u_{xx} + u_{yy} = 0.$$

This is why the real part of a complex function can be used as an exact solution of Laplace's equation.

Important correction:

- checking that the real part is harmonic does **not** prove the whole original complex function is holomorphic,
- it only proves that the extracted real-valued function $u(x, y)$ is harmonic.

To prove a full complex-valued function

$$F(x, y) = u(x, y) + iv(x, y)$$

is holomorphic, the Cauchy–Riemann equations must hold:

$$u_x = v_y, \qquad u_y = -v_x.$$

So for the Laplace project:

- if you only care about the exact real solution for Laplace, checking harmonicity is enough,
- if you want to verify the original whole complex function is holomorphic, you must also check the imaginary part and the Cauchy–Riemann equations.

3 Main SymPy workflow

The basic symbolic pipeline is:

1. define symbols `x`, `y`, `z`,
2. parse user input,
3. substitute `z -> x + I*y`,

4. expand complex structure,
5. extract the real part,
6. simplify it,
7. compute boundary conditions,
8. convert to numeric callable function.

In symbolic form:

$$f(z) \to f(x+iy) \to \Re(f(x+iy)) = u(x,y).$$

4 Why `expand(..., complex=True)` is needed

For simple functions, `as_real_imag()` may work directly.

For more complicated expressions, SymPy may leave the result as something like

$$\text{re(complicated expression)}$$

instead of rewriting it into explicit functions of x and y.

That is why complex expansion is needed first.

Use either:

```
sympy.expand(expr, complex=True)
```

or the clearer equivalent:

```
sympy.expand_complex(expr)
```

This rewrites the expression into explicit real and imaginary parts.

Example:

$$e^{x+iy} = e^x(\cos y + i \sin y)$$

so after expansion the real part becomes obvious:

$$\Re(e^{x+iy}) = e^x \cos y.$$

5 How boundary conditions are computed

Once the real part $u(x,y)$ is known, boundary conditions on the square $[0,1] \times [0,1]$ are found by substitution:

- left: $u(0,y)$,
- right: $u(1,y)$,
- bottom: $u(x,0)$,
- top: $u(x,1)$.

In code this means substituting `x` or `y` with `0` or `1`.

6 Example results that were checked

1. $f(z) = e^z$

$$e^{x+iy} = e^x(\cos y + i \sin y)$$

Real part:

$$u(x,y) = e^x \cos y$$

Boundary conditions:

- left: $u(0,y) = \cos y$
- bottom: $u(x,0) = e^x$
- right: $u(1,y) = e \cos y$
- top: $u(x,1) = e^x \cos 1$

2. $f(z) = \sin z$

$$\sin(x + iy) = \sin x \cosh y + i \cos x \sinh y$$

Real part:

$$u(x, y) = \sin x \cosh y$$

Boundary conditions:

- left: 0
- bottom: $\sin x$
- right: $\sin 1 \cosh y$
- top: $\sin x \cosh 1$

3. $f(z) = z^2$

$$(x + iy)^2 = x^2 - y^2 + 2ixy$$

Real part:

$$u(x, y) = x^2 - y^2$$

Boundary conditions:

- left: $-y^2$
- bottom: x^2
- right: $1 - y^2$
- top: $x^2 - 1$

These tests confirmed that the symbolic pipeline works.

7 Input syntax and user-friendliness

At first, it is better to require SymPy-style input instead of trying to support full WolframAlpha-style math input.

Recommended supported syntax for version 1:

- `exp(z)`
- `sin(z)`
- `cos(z)`
- `z**2`
- `sqrt(z)`
- `1/(1-z)`

Do not start by supporting every possible human-style input like `e^z`.

Later, input can be made friendlier by normalizing it, for example:

- `^` → `**`
- `i` → `I`
- π → `pi`
- maybe `e^z` → `exp(z)` or `E**z`

A good UI idea for the future GUI is to have helper buttons like:

- `sin`, `cos`, `exp`, `sqrt`
- `pi`, `i`, caret symbol, `(`, `)`

This gives the user a "math input mode" feeling without building a full visual formula editor.

8 `subs` is not imported separately

`subs` is not a standalone function to import.

It is a method on SymPy expression objects, so this is the correct use:

`function.subs(z, x + I*y)`

That is why hovering over it in the editor may not show it as a top-level `sympy` import.

9 Typing problems with SymPy and Pylance

A major part of the work was dealing with typing.

The problem

SymPy is highly dynamic, and Pylance/Pyright often reports raw SymPy calls as:

- `Unknown`
- `Basic | Expr`
- `tuple | None`
- partially unknown types

Examples:

- `sympy.expand(...)` is shown as returning `Unknown`,
- `expanded.as_real_imag()` may be shown as returning `tuple | None`,
- `.subs()` may widen type information.

This is mostly a limitation of SymPy typing stubs plus strict Pyright rules, not a problem with the actual math code.

10 Practical typing strategy that worked

1. Use specific types where possible

- `x`, `y`, `z` → `Symbol`
- symbolic expressions → `Expr`
- numeric callable from `lambdify` → `Callable[[float, float], float]`

2. Add a helper to verify and narrow types

A helper like this was used:

```
def require_expr(value: object, name: str) -> Expr:
    if not isinstance(value, Expr):
        raise TypeError(f"{name} is not a SymPy Expr, got {type(value).__name__}")
    return value
```

This gives:

- runtime safety,
- type narrowing for the checker,
- one consistent way to handle SymPy's dynamic return types.

3. Use `cast(object, ...)` at SymPy boundaries when needed

Because some SymPy functions are typed as returning `Unknown`, `cast(object, ...)` was used before passing them to `require_expr`.

This is not cheating; it is a practical boundary solution for a dynamic library.

11 What casting means

Two different meanings were discussed.

Runtime conversion

This changes the actual value:

```
int("5")
float(3)
str(10)
```

Typing cast

This only tells the type checker how to treat the value:

```
from typing import cast
expr_value = cast(Expr, value)
```

Important:

- `cast(...)` does not change the object,
- `cast(...)` does not verify the type,
- it only tells the type checker "trust me".

That is why `require_expr(...)` is safer than only using `cast(...)`.

12 `as_real_imag()` typing issue

Pylance may say:

- the result of `as_real_imag()` is `tuple | None`,
- so indexing directly like `expanded.as_real_imag()[0]` is unsafe.

The correct solution is to split it into steps:

```
parts = expanded.as_real_imag()
if parts is None:
    raise TypeError("as_real_imag() returned None")

real_part = require_expr(parts[0], "real_part")
```

This avoids the "None is not subscriptable" warning.

Also, it is fine not to store the imaginary part if it is unused.

13 Why `Basic` alone was not enough

Using `Basic` is not wrong, but it does not solve the editor warnings by itself.

The issue is not only which type you annotate variables with; the issue is that Pylance sees some SymPy functions themselves as returning `Unknown`.

So the actual practical solution is:

- use `Expr` for project expressions,
- use `require_expr(...)`,
- use `cast(object, ...)` only at problematic SymPy call boundaries.

14 `lambdify` usage

To turn the symbolic real part into a callable numeric function, `sympy.lambdify` is used:

```
u_function = sympy.lambdify((x, y), simplified_real_part, "math")
```

This makes a Python function that can be called numerically, for example:

`u_function(0.5, 0.5)`

A practical type annotation is:

`Callable[[float, float], float]`

This is good enough for the project, even if the exact runtime type may sometimes be int-like.

15 Harmonicity check before using exact solution

This is important.

If the extracted real part is going to be used as the **exact solution** for comparison with a numerical Laplace solver, then it should be checked first.

The check is:

$$\Delta u = u_{xx} + u_{yy}$$

and then simplify it.

In code, this means:

1. compute the second derivative with respect to `x`,
2. compute the second derivative with respect to `y`,
3. add them,
4. simplify the result,
5. check whether it is `0`.

Important distinction:

- if the real part is harmonic, it can be used as an exact Laplace solution,
- if it is not harmonic, its boundary values may still be used to define a numerical boundary-value problem, but it should **not** be used as the exact interior solution for error comparison.

Also important:

- this checks whether the real part is **harmonic**,
- it does **not** prove the original complex function is holomorphic.

16 Correct comment for the code

The code comment should say something like:

- `# Checking if the real part u(x,y) is harmonic`

It should **not** say:

- `# Checking if the function f(z) is holomorphic`

because that is mathematically different.

17 Final design recommendation for the project

The SymPy logic is already strong enough to integrate into the larger Laplace project.

Recommended architecture:

- keep SymPy processing in its own module,
- let the rest of the project call that module.

For example, the symbolic module can handle:

- parsing,
- substitution,
- complex expansion,
- real-part extraction,

- simplification,
- boundary-condition generation,
- harmonicity checking,
- numeric function generation.

This keeps the rest of the project separate from SymPy internals.

18 What is ready now

The current implementation is already enough for first integration.

It can:

- parse a complex function,
- compute the real part,
- compute boundary conditions,
- evaluate numerically,
- check harmonicity.

That is enough to merge into the larger Laplace project.

19 Suggested next steps

1. move the SymPy logic into separate functions or a separate module,
2. add `try/except` around parsing and symbolic processing,
3. restrict supported input syntax for version 1,
4. test more expressions such as:
   - `exp(z)`
   - `sin(z)`
   - `z**2`
   - `1/(1-z)`
5. later improve input normalization,
6. only after that, integrate into a GUI such as PySide6.

20 One-sentence conclusion

You now have a working and reasonably well-typed SymPy pipeline that can convert a user-provided complex function into a real harmonic candidate, compute its boundary conditions on $[0,1] \times [0,1]$, evaluate it numerically, and verify whether it is valid as an exact Laplace solution.